Statistics Basic

LAB 10 - [PBT] 선형회귀

1. 분석용 데이터 전처리

ITWILL 이광호 강사

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



데이터 분석 프로세스

□ 앞에서 살펴본 주요 분석 기법을 다시 한번 점검하고 모델링을 수행합니다.

문제정의	데이터 수집	데이터 품질 점검 및 기본 전처리	탐색적 데이터 분석	모델링용 데이터 전처리	모델 구축	모델 평가 및 개선	결과 해석 및 활용
분석 배경 및 동기 사회적·학문적 필요성 선행연구 검토	**데이터 확보** 내부 데이터, 외부 데이터 데이터 출처 확인	**데이터 구조 점검** 데이터 재점검 (크기, 내용) 데이터 타입 점검 중복값 점검 행·열 이름 점검 행·열 재배치 행·열 추가/삭제	**시각화** 분포·패턴·관계 확인	**통계적 기법 활용** 파생변수 생성 (파생변수 기술 통계량 확인 필요) 공간특성 반영 명목형 라벨링 더미변수 인코딩 로그 변환 결측치,이상치 대체 다중 공선성 확인 데이터 정규화	모형 적합 회귀계수 추정 가정 검토 (정규성, 등분산성, 독립성)	적합도 평가 유의성 검토 잔차 진단 모형 수정	**결과 해석** 영향 해석 인사이트 도출
분석 목표 설정 문제 확인 분석 질문 구체화 의사결정지원범위	**데이터 구조 확인** 행·열 크기 변수명 확인 데이터 타입 확인	**데이터 품질 점검** 형식·표기 · 단위 점검 결측치 점검 (삭제만 수행)	**추론통계** 상관분석(연속형) 교차분석(명목형) 차이 검토 (T-Test) 관계 검토 (ANOVA)				**결과 보고** 표 그래프 보고서
문제 유형 결정 예측, 분류 군집화, 추천 패턴 탐지	**데이터 이해** 데이터 의미 파악 데이터 단위 파악	**기술통계량 확인** 중심수준 파악 퍼짐과 안정성 판단 비대칭 신호 확인 이상치 점검 (삭제만 수행) 기술 통계량 해석	**가설 생성** 패턴 기반 연구 가설 도출				**연구의 의의** 학문적·실무적 시사점 활용 방안 제언
분석 범위 설정 시간적 범위 공간적 범위	**데이터 한계 인지** 표본 편향 수집 범위 제한			**머신러닝 기법 활용** 파생변수 생성 (파생변수 기술 통계량 확인 필요) 공간특성 반영 로그 변환 학습·검증용 데이터 분할	데이터 라벨링 더미변수 인코딩 결측치,이상치 대체 다중공선성 확인 스케일링 차원축소 베이스 모델 구축 (단순 모델 비교)	최적 모델 선정 교차검증 및 하이퍼파라미터 튜닝 과적합 점검	**한계 및 과제** 데이터·방법 한계점 후속 연구 제안
변수 정의 목표/설명 변수 가설 수립							

공간특성 반영은 군집분석(머신러닝) 이후에 다룹니다.

ITWILL 이광호강사



전처리 프로세스

□ 전처리는 각 단계가 다음 단계에 미치는 영향으로 결정된다

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5	STEP 6
라벨링	더미변수	로그변환	이상치 대체	다중공선성	정규화

로그 변환 후 CRIM 이상치 66개(13.0%) → 6개(1.2%)

□ 전처리 체크 포인트

◎ 각 단계마다 데이터를 저장해서 각각 모델링을 수행 후 성능을 비교한다.

체크포인트	시점	비교 의미
0. 기준선 ※ 필수 변환 완료 지점	라벨링 + 더미	아무 선택 전처리도 안 한 출발점. 이후 모든 성능 차이의 기준이 된다.
1. 로그변환	+ 로그	분포를 펴면 성능이 오르는가
2. 이상치 대체	+ 이상치	극단값을 눌러야 계수가 안정되는가
3. 다중공선성 제거	+ VIF	변수를 덜어내도 성능이 유지되는가
4. 정규화	+ 정규화	최종 전처리본 — 모든 단계 적용

ITWILL 이광호강사



boston 데이터셋의 독립변수 선별표

✅ 채택 5 · 🟡 후보 8 · ❌ 제외 0

□ EDA에 따른 결론

◎ 모델링 과정에서 유의하지 않은 변수는 탈락할 것이므로 후보 변수는 제거하지 않는다

채택여부	변수	검정방법	유의수준 (p)	효과크기	가정점검	비대칭신호	공선성 신호	근거
✅ 채택	LSTAT	Spearman	<0.001	ρ=-0.853 (강함)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 없음 / 고왜도(→Spearman)	우편향 / log1p 필요	❌	유의미한 상관 (|coef|≥0.3)
✅ 채택	RM	Spearman	<0.001	ρ=+0.634 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 없음 / 고왜도(→Spearman)	대칭 / 불필요	❌	유의미한 상관 (|coef|≥0.3)
✅ 채택	PTRATIO	Spearman	<0.001	ρ=-0.556 (중간)	정규성 위배 / 선형성 충족 / 이상치 계수왜곡 없음 / 고왜도(→Spearman)	좌편향 / 불필요	❌	유의미한 상관 (|coef|≥0.3)
✅ 채택	ZN	Spearman	<0.001	ρ=+0.438 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 없음 / 고왜도(→Spearman)	우편향 / log1p 필요	❌	유의미한 상관 (|coef|≥0.3)
✅ 채택	CHAS	Mann-Whitney U test	0.002	미제공 (U=5605.5)	정규성 위배	해당 없음	❌	집단 간 차이 유의함
🟡 후보	INDUS	Spearman	<0.001	ρ=-0.578 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	대칭 / 불필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	NOX	Spearman	<0.001	ρ=-0.563 (중간)	정규성 위배 / 선형성 충족 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	우편향 / 불필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	TAX	Spearman	<0.001	ρ=-0.562 (중간)	정규성 위배 / 선형성 충족 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	우편향 / 불필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	CRIM	Spearman	<0.001	ρ=-0.559 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 없음 / 고왜도(→Spearman)	우편향 / log1p 필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	AGE	Spearman	<0.001	ρ=-0.548 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	좌편향 / 불필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	DIS	Spearman	<0.001	ρ=+0.446 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	우편향 / log1p 필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	RAD	Spearman	<0.001	ρ=-0.347 (중간)	정규성 위배 / 선형성 위배 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	우편향 / log1p 필요	⭕	유의미한 상관 (|coef|≥0.3) · 공선성 존재 → 후보(대표변수 선택 필요)
🟡 후보	B	Spearman	<0.001	ρ=+0.186 (약함)	정규성 위배 / 선형성 충족 / 이상치 계수왜곡 있음 / 고왜도(→Spearman)	좌편향 / reverse_log1p 필요	❌	유의하나 관계 약함 (|coef|<0.3)

ITWILL 이광호강사

1. 모델링용 전처리
2. Boston Housing
회귀 모델 적합
3. Diamond Price
Prediction
회귀 모델 적합



#01. 준비작업

실습

1. 이전 작업 내역 불러오기

```
1 %%capture cap
2 %run "./[LAB-08] PBT - EDA | 1-EDA 시작하기(실습코드).ipynb"
```

✓ 마지막 실행: 오전 9:34:19, 기간: 2.4s Python

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



#01. 준비작업

2. 불러온 내용 확인

```
print("목표변수 :", target)
print("연속형(종속변수 포함) :", continuous_cols)
print("독립변수(명목형) :", nominal_cols)
print("독립변수(연속형) :", feature_continuous)

display(df.head())          # 품질점검 완료 후 타입변환 데이터 프레임
display(desc)               # 기술통계량 데이터 프레임
display(cat_desc)           # 범주형 데이터 기술통계량 데이터 프레임
```

✓ 마지막 실행: 오전 9:34:19, 기간: 0.0s Python

```
목표변수 : MEDV
연속형(종속변수 포함) : ['CRIM', 'ZN', 'INDUS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']
독립변수(명목형) : ['CHAS']
독립변수(연속형) : ['CRIM', 'ZN', 'INDUS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.006	18.000	2.310	0	0.538	6.575	65.200	4.090	1	296	15.300	396.900	4.980	24.000
1	0.027	0.000	7.070	0	0.469	6.421	78.900	4.967	2	242	17.800	396.900	9.140	21.600
2	0.027	0.000	7.070	0	0.469	7.185	61.100	4.967	2	242	17.800	392.830	4.030	34.700
3	0.032	0.000	2.180	0	0.458	6.998	45.800	6.062	3	222	18.700	394.630	2.940	33.400
4	0.069	0.000	2.180	0	0.458	7.147	54.200	6.062	3	222	18.700	396.900	5.330	36.200

	count	mean	std	min	25%	50%	75%	max	rel_diff	rdiff_flag	iqr	upper_bound	lower_bound	up
CRIM	506.000	3.614	8.602	0.006	0.082	0.257	3.677	88.976	13.087	large_diff	3.595	9.070	-5.311	
ZN	506.000	11.364	23.322	0.000	0.000	0.000	12.500	100.000	inf	large_diff	12.500	31.250	-18.750	
INDUS	506.000	11.137	6.860	0.460	5.190	9.690	18.100	27.740	0.149	diff	12.910	37.465	-14.175	
NOX	506.000	0.555	0.116	0.385	0.449	0.538	0.624	0.871	0.031	similar	0.175	0.886	0.187	
RM	506.000	6.285	0.703	3.561	5.885	6.208	6.623	8.780	0.012	similar	0.738	7.731	4.778	

... 생략 ...



파생 변수의 이해

boston 데이터셋에는 해당 없음

□ 기존 변수(원시 데이터)를 가공·결합·변환하여 새롭게 생성한 변수.

◎ 모델의 예측력과 데이터의 해석력을 높이는 것을 목적으로 함.

○ 분석가의 주관에 따라 생성 여부를 결정한다 (도메인 지식 필요)

□ 생성 프로세스

STEP 1 원시 데이터 → STEP 2 변수 선택 → STEP 3 변환 · 결합 → STEP 4 파생변수 생성

□ 주요 생성 기법

1. **산술 결합 (Arithmetic)**
   예) BMI = 체중 / 키^2
2. **비율 · 비교 (Ratio)**
   예) 부채비율 = 부채 / 자산
3. **시간 파생 (Time-based)**
   예) 가입일 → 가입경과일, 요일, 계절
4. **구간화 (Binning)**
   예) 연령 → 10대 · 20대 · 30대 …

□ 기대효과

비선형 관계 표현
단일 변수로 포착 불가능한 패턴을 표현

모델 예측력 향상
정보 밀도를 높여 학습 효율 개선

도메인 지식 반영
전문가 통찰을 수치적 변수로 구현

ITWILL 이광호강사



데이터 라벨링 (라벨 인코딩)

□ 명목형 범주형 변수의 각 범주를 고유한 정수(0, 1, 2, ...)로 매핑하는 변환 기법

왜 필요한가?

- 통계 • 머신러닝 모델은 텍스트 데이터를 직접 처리하기 어려움
- 범주형 데이터를 계산 가능한 형태로 만들기 위하여 수치형으로 변환

✓ 텍스트 → 숫자 변환으로 분석 가능성 확보

변환 방식

- 각 범주에 고유한 정수 라벨을 1:1로 대응시킴

[빨강, 파랑, 초록] 원본 데이터 → **[0, 1, 2]** 라벨 인코딩

텍스트 범주 → 정수 라벨 변환

예시: 색상 범주 변환

범주값 (전)	라벨값 (후)	설명
빨강	0	첫 번째 범주
파랑	1	두 번째 범주
초록	2	세 번째 범주

주의사항

- **숫자의 크기나 순서가 실제 의미를 가지는 것은 아니다.**
- **예를 들어, "파랑(1) > 빨강(0)"은 색상의 우선순위를 의미하는 것이 아니다.**

ITWILL 이광호강사

1. 모델링용 전처리
2. Boston Housing
회귀 모델 적합
3. Diamond Price
Prediction
회귀 모델 적합



/helpers/my_prep.py (step-1)

□ 라이브러리 참조

◎ 이 단원에서 사용될 라이브러리를 미리 모두 참조해 둔다.

```
1  import os
2  import joblib
3  import numpy as np
4  from pandas import pivot_table, get_dummies
5  from scipy.stats import skew
6  from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler, MaxAbsScaler
7  from sklearn.preprocessing import LabelEncoder
8  from . import my_stats
9
10 # scaling() 에서 사용할 스케일러 이름과 클래스의 매핑
11 SCALERS = {
12     'standard': StandardScaler,
13     'minmax': MinMaxScaler,
14     'robust': RobustScaler,
15     'maxabs': MaxAbsScaler,
16 }
```

1. 모델링용 전처리
2. Boston Housing
회귀 모델 적합
3. Diamond Price
Prediction
회귀 모델 적합



/helpers/my_prep.py (step-2)

□ 데이터 라벨링 함수 정의

```python
def labeling(df, columns, save_path=None, verbose=True):
    """
    지정한 범주형 컬럼들의 값을 0부터 시작하는 정수로 변환하는 함수

    Args:
        df (DataFrame): 라벨링을 적용할 데이터프레임
        columns (list): 라벨링할 컬럼명 리스트
        save_path (str, optional): 학습된 인코더들을 저장할 pkl 파일 경로
            (예: 'models/encoders.pkl') (기본값: None, 저장하지 않음)
        verbose (bool): 컬럼별로 원래 값이 어떤 정수에 대응되는지 출력할지 여부 (기본값: True)

    Returns:
        DataFrame: 라벨링이 적용된 데이터프레임 (원본은 변경되지 않는다)
    """
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-3)

실습

□ 작업 파이프라인 구성

```
150 def labeling(df, columns, save_path=None, verbose=True):
151     """…
164     # --- 1) 작업 준비 ---
165
166     # --- 2) 컬럼별 라벨 인코딩 (문자열 -> 0부터 시작하는 정수) ---
167
168     # --- 3) 변환 내역 출력 (원래 값 -> 부여된 정수) ---
169
170     # --- 4) 학습된 인코더 저장 (선택) ---
171
172     # --- 5) 라벨링이 적용된 데이터프레임 반환 ---
173
```



/helpers/my_prep.py (step-4)

□ 라벨링 처리

```
164     # --- 1) 작업 준비 ---
165     result = df.copy()     # 원본을 보존하기 위해 복사본으로 작업
166     encoders = {}          # 역변환과 test 데이터 적용을 위해 컬럼별 인코더를 보관
167
168     # --- 2) 컬럼별 라벨 인코딩 (문자열 -> 0부터 시작하는 정수) ---
169     for c in columns:
170         encoder = LabelEncoder()
171         result[c] = encoder.fit_transform(df[c])
172         encoders[c] = encoder
173
174     # --- 3) 변환 내역 출력 (원래 값 -> 부여된 정수) ---
175     if verbose:
176         for c in columns:
177             # classes_ 의 순서가 곧 부여된 정수값이므로 짝지어 출력한다
178             pairs = []
179             for i, v in enumerate(encoders[c].classes_):
180                 pairs.append(f'{v}={i}')
181
182             print(f'{c} ({len(pairs)}종): {", ".join(pairs)}')
```



/helpers/my_prep.py (step-5)

□ 인코더 저장 및 결과 반환

◎ 추후 라벨링된 값을 원래의 값으로 되돌려야 할 경우를 대비하여
라벨링 정보를 담고 있는 객체를 파일로 저장한다. → 머신러닝이므로 뒤에서 설명합니다

```
184     # --- 4) 학습된 인코더 저장 (선택) ---
185     # 컬럼마다 인코더가 따로 있으므로 dict 통째로 하나의 pkl 파일에 저장한다
186     if save_path:
187         folder = os.path.dirname(save_path)
188         if folder:
189             # 경로에 없는 폴더가 있으면 만들어 준다
190             os.makedirs(folder, exist_ok=True)
191 
192         joblib.dump(encoders, save_path)
193 
194         if verbose:
195             print(f'\n인코더 저장: {save_path} ({len(encoders)}개 컬럼)')
196 
197     # --- 5) 라벨링이 적용된 데이터프레임 반환 ---
198     return result
```



#02. 데이터 전처리

1. 명목형 라벨링

◎ 명목형 컬럼(CHAS)은 category 타입이라 그대로는 모델에 넣을 수 없으므로 정수로 바꾼다.

```
1 df1 = my_prep.labeling(df, columns=nominal_cols)
2 df1.head()
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

```
CHAS (2종): 0=0, 1=1
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.006	18.000	2.310	0	0.538	6.575	65.200	4.090	1	296	15.300	396.900	4.980	24.000
1	0.027	0.000	7.070	0	0.469	6.421	78.900	4.967	2	242	17.800	396.900	9.140	21.600
2	0.027	0.000	7.070	0	0.469	7.185	61.100	4.967	2	242	17.800	392.830	4.030	34.700
3	0.032	0.000	2.180	0	0.458	6.998	45.800	6.062	3	222	18.700	394.630	2.940	33.400
4	0.069	0.000	2.180	0	0.458	7.147	54.200	6.062	3	222	18.700	396.900	5.330	36.200



명목형 변수의 One-Hot-Encoding (더미변수)

□ 명목형 데이터의 수치형 데이터 변환

◎ 라벨링만 적용하면 모델이 숫자의 크기에 잘못된 순서·서열을 학습할 수 있기 때문에 독립적인 더미변수로 변환해 주는 처리

◎ 통계적 용어 : **더미변수** / 머신러닝 용어 : **One-Hot-Encoding** (혹은 Encoding)

성별
남자
여자

숫자로 값을 구분 → 라벨링 →

성별
0
1

라벨링된 값은 컬럼명의 접미사가 됨 → 인코딩 →

성별_0	성별_1
1	0
0	1

더미변수에서 1은 True, 0은 False의 의미

□ 라벨링을 생략한 One-Hot-Encoding

◎ 인코딩을 위해 라벨링이 반드시 수행되어야 하는 것은 아님

◎ 하지만 라벨링을 거치지 않을 경우 변수 이름이 지나치게 길고 복잡해지는 경우가 있음

성별
남자
여자

→ 인코딩 →

성별_남자	성별_여자
1	0
0	1

pandas 패키지를 사용하는 방법과 scikit-learn 패키지를 사용하는 방법이 있다.
- 통계: pandas 사용 / 머신러닝 : scikit-learn 사용

ITWILL 이광호강사



더미 변수 처리의 drop_first=True

□ 이진 변수 → 값이 0과 1로만 구성되어 있는 경우

◎ **성별_0이 1**이라면(성별이 남자라면), **성별_1은 반드시 0**(여자가 아님)이 된다.

○ 라벨링을 수행한 이진 변수의 더미 변수 처리(drop_first=True)는 의미 없는 처리가 됨

성별
남자
여자

→ 라벨링 →

성별
0
1

→ 인코딩 (drop_first=True) →

성별_0	성별_1
~~1~~	0
~~0~~	1

□ 값의 종류가 3개 이상인 경우

◎ 더미 변수 처리는 값의 종류별로 변수(컬럼)를 만들고
이 중 **하나가 1이라면 나머지 항목은 필연적으로 0이 되므로**
첫 번째 항목을 제거하더라도 충분히 추론이 가능하다.

○ **합격 여부가 보류인 경우 합격여부_0이 1이므로 나머지는 반드시 0이 된다.**

합격여부
보류
불합격
합격

→ 라벨링 →

합격여부
0
1
2

→ 인코딩 (drop_first=True) →

합격여부_0	합격여부_1	합격여부_2
1	**0**	**0**
~~0~~	1	0
~~0~~	0	1

이진변수(값의 종류가 두 개인 범주형)는 라벨링만 수행

값의 종류가 3개 이상인 범주형은 라벨링 + 더미처리

ITWILL 이광호강사

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-6)

□ 더미변수 생성 함수 정의

```
284 def dummies(df, columns, drop_first=True, verbose=True):
285     """
286     지정한 범주형 컬럼들을 값의 종류마다 0/1 컬럼으로 펼치는 함수
287 
288     Args:
289         df (DataFrame): 더미 변환을 적용할 데이터프레임
290         columns (list): 더미 변환할 컬럼명 리스트
291         drop_first (bool): 각 컬럼에서 만들어진 더미 중 첫 번째를 제외할지 여부 (기본값: True)
292             k개의 값에서 k개의 더미를 모두 만들면 서로의 합이 항상 1이 되어 완전한
293             다중공선성(더미 변수 함정)이 생기므로, 하나를 빼서 기준(reference) 범주로 삼는다
294         verbose (bool): 컬럼별로 생성된 더미 컬럼과 생략된 컬럼을 출력할지 여부 (기본값: True)
295 
296     Returns:
297         DataFrame: 더미 변환이 적용된 데이터프레임 (원본은 변경되지 않는다)
298     """
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-7)

□ 작업 파이프라인 구성

```
284  def dummies(df, columns, drop_first=True, verbose=True):
285      """…
299      # --- 1) 처리 대상 컬럼 결정 ---
300
301      # --- 2) 더미 변환 수행 ---
302
303      # --- 3) 변환 내역 출력 (생성된 더미 컬럼과 생략된 컬럼) ---
304
305      # --- 4) 더미 변환이 적용된 데이터프레임 반환 ---
306
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-8)

□ 더미변수 처리하기

```
299     # --- 1) 처리 대상 컬럼 결정 ---
300     # 값의 종류가 2개인 컬럼은 변환 대상에서 제외한다
301     targets = []
302     skipped = []
303
304     for c in columns:
305         if df[c].nunique() == 2:
306             skipped.append(c)
307         else:
308             targets.append(c)
309
310     # --- 2) 더미 변환 수행 ---
311     # dtype=int 를 지정해 True/False 가 아닌 0/1 로 만든다
312     result = get_dummies(df, columns=targets, drop_first=drop_first, dtype=int)
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-9)

□ 변환 내역 출력 및 결과 반환

```python
    # --- 3) 변환 내역 출력 (생성된 더미 컬럼과 생략된 컬럼) ---
    if verbose:
        for c in targets:
            # 원본에 없고 결과에만 있으면서 'c_' 로 시작하는 컬럼이 c 로부터 생성된 더미다
            created = []
            for new in result.columns:
                if new not in df.columns and new.startswith(f'{c}_'):
                    created.append(new)

            # drop_first 로 빠진 기준 범주가 무엇인지 함께 알려준다
            dropped = ''
            if drop_first:
                dropped = f'  (기준: {sorted(df[c].unique())[0]} 제외)'

            print(f'{c} ({df[c].nunique()}종) -> {len(created)}개: {", ".join(created)}{dropped}')

        for c in skipped:
            print(f'{c} (2종) -> 생략: 이진 변수이므로 원래 컬럼을 유지')

        print(f'\n컬럼 수: {df.shape[1]}개 -> {result.shape[1]}개')

    # --- 4) 더미 변환이 적용된 데이터프레임 반환 ---
    return result
```



#02. 데이터 전처리

2. 더미변수 인코딩

◎ 라벨링된 명목형 컬럼을 더미 변수로 펼친다.

○ 이 과정까지는 **모델에 넣기 위한 필수 변환**이므로, 이 시점의 데이터프레임을 기준선으로 저장한다

```
1 df2 = my_prep.dummies(df1, columns=nominal_cols, drop_first=True)
2 df2.to_excel("체크포인트0_기준선.xlsx", index=False)
3 df2.head()
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.2s Python

CHAS (2종) -> 생략: 이진 변수이므로 원래 컬럼을 유지

컬럼 수: 14개 -> 14개

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.006	18.000	2.310	0	0.538	6.575	65.200	4.090	1	296	15.300	396.900	4.980	24.000
1	0.027	0.000	7.070	0	0.469	6.421	78.900	4.967	2	242	17.800	396.900	9.140	21.600
2	0.027	0.000	7.070	0	0.469	7.185	61.100	4.967	2	242	17.800	392.830	4.030	34.700
3	0.032	0.000	2.180	0	0.458	6.998	45.800	6.062	3	222	18.700	394.630	2.940	33.400
4	0.069	0.000	2.180	0	0.458	7.147	54.200	6.062	3	222	18.700	396.900	5.330	36.200

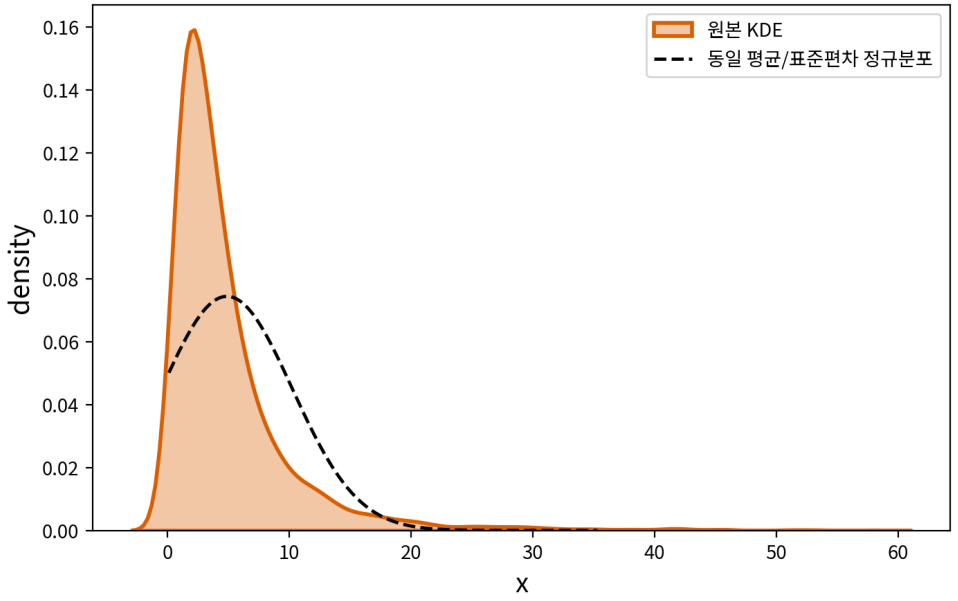
로그 변환의 이해

□ 오른쪽으로 치우친 양수 데이터를 더 대칭적인 형태로 완화하기 위해 사용

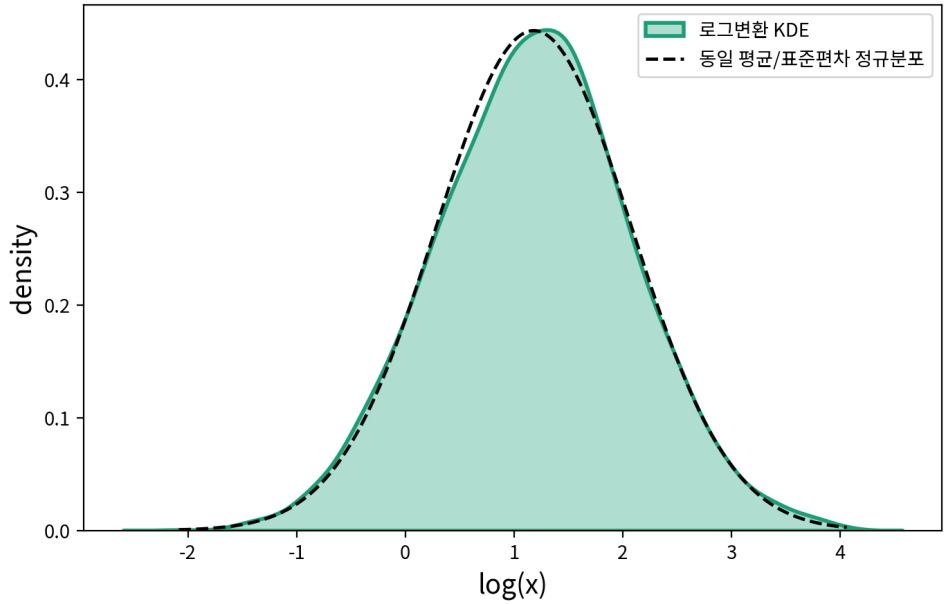
◎ 대표적으로 소득, 매출, 인구, 거래금액, 주택가격처럼 큰 값이 일부 존재하는 변수에 자주 적용

◎ 큰 값의 상대적 영향을 줄이는 방법

원본 데이터: 강한 우측 왜도



로그 변환 후: 정규분포에 근접



- **자연 로그는 0을 처리하지 못하기 때문에 데이터에 0이 포함되어 있으면, +1을 적용한 후 로그 변환을 해야 한다.**
- **우측 꼬리 분포에만 적용해야 한다.**
- **좌측 꼬리 분포의 경우 데이터를 반전(최대값 - 원데이터) 처리 한 후 로그 변환을 수행한다.**

ITWILL 이광호강사



로그 변환 방법 (1)

□ 데이터가 우측 꼬리인 경우

◎ 0을 포함하지 않는 변수에 대한 로그 변환 (자연로그)

```
df['변수명'] = np.log(df['변수명'])
```

* logarithm

◎ 자연로그 변환된 데이터를 원래 값으로 역변환

```
df['변수명'] = np.exp(df['변수명'])
```

* exponential

◎ 원본 데이터에 1을 더한 뒤 로그를 취하기 (권장)

○ 자연로그는 0에 대한 처리가 불가능하지만 1을 더하면 0에 대한 처리도 가능함.

```
df['변수명'] = np.log1p(df['변수명'])
```

◎ log1p에 대한 역변환

```
df['변수명'] = np.expm1(df['변수명'])
```



로그 변환 방법 (2)

$$z = \log(1 + (x_\max - x))$$

최대값 - 원데이터로 값을 반전한 뒤,
0이 나올 수 있으므로 1을 더해서
로그 변환함.

□ 데이터가 좌측 꼬리인 경우

◎ 로그 변환

○ 값을 반전한 뒤 로그 변환을 수행해야 함 (최대값 - 원본값)

```
x_max = df['변수명'].max()
df['변수명'] = np.log1p(x_max - df['변수명'])
```

NumPy Code

```
z = np.log1p(x_max - x)
```

◎ 원래 데이터로 역변환

○ 역변환에 사용한 x_max를 그대로 사용해야 함

```
df['변수명'] = x_max - np.expm1(df['변수명'])
```

역변환을 위해 양쪽에 exp를 적용하면

$$exp(z) = 1 + (x_max - x)$$

양쪽에서 1을 빼면

$$exp(z) - 1 = x_max - x$$

즉,

$$expm1(z) = x_max - x$$

따라서

$$x = x_max - expm1(z)$$

NumPy Code

```
x_original = x_max - np.expm1(z)
```


ITWILL 이광호강사



/helpers/my_prep.py (step-10)

□ 로그 변환 함수 정의

```
43 def log_transform(df, log_columns=None, reflect_columns=None, verbose=True):
44     """
45     로그변환 함수
46 
47     Args:
48         df (DataFrame): 변환을 적용할 데이터프레임
49         log_columns (list, optional): 로그 변환할(우측 꼬리) 컬럼명 리스트 (기본값: None)
50         reflect_columns (list, optional): 반사 후 로그 변환할(좌측 꼬리) 컬럼명 리스트
           (기본값: None)
51         verbose (bool): 컬럼별 변환식·역변환식과 왜도 변화를 출력할지 여부 (기본값: True)
52 
53     Returns:
54         DataFrame: 변환이 적용된 데이터프레임 (원본은 변경되지 않는다)
55     """
```



/helpers/my_prep.py (step-11)

□ 구현 단계 파이프라인

```
45  def log_transform(df, log_columns=None, reflect_columns=None, verbose=True):
46      """…
58      # --- 1) 작업 준비 ---
59
60      # --- 2) 우측 꼬리 컬럼 변환: log(1+x) ---
61      # 값이 0인 경우 log(0) = -inf 가 되므로 log(1+x) 를 사용한다
62
63      # --- 3) 좌측 꼬리 컬럼 변환: 반사 후 log(1+x) ---
64
65      # --- 4) 변환 내역 출력 (변환식·역변환식과 왜도 변화) ---
66
67      # --- 5) 변환이 적용된 데이터프레임 반환 ---
68
69
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-12)

□ 작업 준비 및 우측 꼬리에 대한 로그 변환

```
58      # --- 1) 작업 준비 ---
59      result = df.copy()      # 원본을 보존하기 위해 복사본으로 작업
60      report = []             # verbose 출력을 위해 컬럼별 변환 내역을 기록
61
62      # --- 2) 우측 꼬리 컬럼 변환: log(1+x) ---
63      # 값이 0인 경우 log(0) = -inf 가 되므로 log(1+x) 를 사용한다
64      if log_columns:
65          for c in log_columns:
66              result[c] = np.log1p(df[c])
67              report.append([c, '우측 꼬리', 'log(1+x)', 'exp(y)-1'])
```

1. 모델링용 전처리
2. Boston Housing
회귀 모델 적합
3. Diamond Price
Prediction
회귀 모델 적합



/helpers/my_prep.py (step-13)

□ 좌측꼬리 변환

```
69      # --- 3) 좌측 꼬리 컬럼 변환: 반사 후 log(1+x) ---
70      # 최댓값에서 빼면 좌우가 뒤집혀 좌측 꼬리가 우측 꼬리가 되므로, 그 뒤 동일하게 로그를 취한다
71      # 값의 대소 관계가 뒤집히므로 회귀계수의 부호도 반대로 해석해야 한다
72      if reflect_columns:
73          for c in reflect_columns:
74              # 역변환하려면 이 최댓값이 반드시 필요하므로 verbose 출력에 함께 남긴다
75              max_value = df[c].max()
76              result[c] = np.log1p(max_value - df[c])
77              report.append([c, '좌측 꼬리',
78                             f'log(1+{max_value:g}-x)',
79                             f'{max_value:g}-(exp(y)-1)'])
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-14)

□ 로그 변환 내역 출력 및 결과 반환

◎ 추후 원래 값으로 역변환을 수행하려면 좌측꼬리 변환에서 사용된 최대값을 기록해 두어야 한다.

```
81      # --- 4) 변환 내역 출력 (변환식·역변환식과 왜도 변화) ---
82      if verbose:
83          print(f'{"컬럼":10s}{"꼬리방향":10s}{"변환식":22s}{"역변환식":24s}{"왜도":>16s}')
84          print('-' * 88)
85
86          for c, side, func, inverse in report:
87              before = skew(df[c].dropna())
88              after = skew(result[c].dropna())
89              change = f'{before:.2f} -> {after:.2f}'
90              print(f'{c:10s}{side:10s}{func:22s}{inverse:24s}{change:>14s}')
91
92      # --- 5) 변환이 적용된 데이터프레임 반환 ---
93      return result
```



/helpers/my_prep.py (step-15)

□ 로그 역변환 함수 정의

```
100 def inverse_log_transform(df, log_columns=None, reflect_columns=None, verbose=True):
101     """
102     log_transform() 으로 변환된 컬럼을 원래 값(단위)으로 되돌리는 함수
103 
104     Args:
105         df (DataFrame): 역변환을 적용할 데이터프레임
106         log_columns (list, optional): 로그 변환했던(우측 꼬리) 컬럼명 리스트 (기본값: None)
107         reflect_columns (dict, optional): 반사 변환했던(좌측 꼬리) 컬럼의
108             {컬럼명: 변환 당시의 최댓값} (예: {'B': 396.9}) (기본값: None)
109         verbose (bool): 컬럼별 역변환식과 값의 범위 변화를 출력할지 여부 (기본값: True)
110 
111     Returns:
112         DataFrame: 역변환이 적용된 데이터프레임 (원본은 변경되지 않는다)
113     """
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-16)

실습

□ 작업 파이프라인

```
97  # ============================================================
98  # 로그 역변환
99  # ============================================================
100 def inverse_log_transform(df, log_columns=None, reflect_columns=None, verbose=True):
101     """…
114     # --- 1) 작업 준비 ---
115
116     # --- 2) 우측 꼬리 컬럼 역변환: exp(y)-1 ---
117
118     # --- 3) 좌측 꼬리 컬럼 역변환: 최댓값 - (exp(y)-1) ---
119
120     # --- 4) 역변환 내역 출력 (역변환식과 값의 범위 변화) ---
121
122     # --- 5) 역변환이 적용된 데이터프레임 반환 ---
123
```



/helpers/my_prep.py (step-17)

□ 작업 준비 및 우측, 좌측 꼬리 역변환

```
114     # --- 1) 작업 준비 ---
115     result = df.copy()     # 원본을 보존하기 위해 복사본으로 작업
116     report = []            # verbose 출력을 위해 컬럼별 역변환 내역을 기록
117
118     # --- 2) 우측 꼬리 컬럼 역변환: exp(y)-1 ---
119     # log(1+x) 의 역함수인 exp(y)-1 로 되돌린다
120     if log_columns:
121         for c in log_columns:
122             result[c] = np.expm1(df[c])
123             report.append([c, '우측 꼬리', 'exp(y)-1'])
124
125     # --- 3) 좌측 꼬리 컬럼 역변환: 최댓값 - (exp(y)-1) ---
126     # 로그를 먼저 풀고(exp(y)-1), 그 결과를 최댓값에서 빼서 뒤집힌 대소 관계를 되돌린다
127     if reflect_columns:
128         for c, max_value in reflect_columns.items():
129             result[c] = max_value - np.expm1(df[c])
130             report.append([c, '좌측 꼬리', f'{max_value:g}-(exp(y)-1)'])
```



/helpers/my_prep.py (step-18)

□ 변환 내역 출력 및 결과 반환

```
132    # --- 4) 역변환 내역 출력 (역변환식과 값의 범위 변화) ---
133    if verbose:
134        print(f'{"컬럼":10s}{"꼬리방향":10s}{"역변환식":24s}{"값의 범위":>28s}')
135        print('-' * 76)
136
137        for c, side, inverse in report:
138            before = f'{df[c].min():.2f}~{df[c].max():.2f}'
139            after = f'{result[c].min():.2f}~{result[c].max():.2f}'
140            change = f'{before} -> {after}'
141            print(f'{c:10s}{side:10s}{inverse:24s}{change:>26s}')
142
143    # --- 5) 역변환이 적용된 데이터프레임 반환 ---
144    return result
```



#02. 데이터 전처리

3. 로그 변환 대상 컬럼 선정

```python
# 우측꼬리 로그 변환 대상 컬럼
log_cols = desc.loc[desc['log_need'] == 'log1p'].index.tolist()
log_cols
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

```
['CRIM', 'ZN', 'DIS', 'RAD', 'LSTAT', 'MEDV']
```

```python
# 좌측꼬리 로그 변환 대상 컬럼
reflect_cols = desc.loc[desc['log_need'] == 'reverse_log1p'].index.
tolist()
reflect_cols
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

```
['B']
```



#02. 데이터 전처리

4. 로그 변환

```
1 df3 = my_prep.log_transform(df2, log_columns=log_cols,
2                             reflect_columns=reflect_cols)
3 df3.to_excel("체크포인트1_로그변환.xlsx", index=False)
4 df3.head()
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

```
컬럼        꼬리방향        변환식              역변환식                  왜도
-------------------------------------------------------------------------------
CRIM        우측 꼬리       log(1+x)            exp(y)-1              5.21 -> 1.27
ZN          우측 꼬리       log(1+x)            exp(y)-1              2.22 -> 1.19
DIS         우측 꼬리       log(1+x)            exp(y)-1              1.01 -> 0.33
RAD         우측 꼬리       log(1+x)            exp(y)-1              1.00 -> 0.53
LSTAT       우측 꼬리       log(1+x)            exp(y)-1             0.90 -> -0.19
MEDV        우측 꼬리       log(1+x)            exp(y)-1             1.10 -> -0.24
B           좌측 꼬리       log(1+396.9-x)      396.9-(exp(y)-1)     -2.88 -> 0.60
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.006	2.944	2.310	0	0.538	6.575	65.200	1.627	0.693	296	15.300	0.000	1.788	3.219
1	0.027	0.000	7.070	0	0.469	6.421	78.900	1.786	1.099	242	17.800	0.000	2.316	3.118
2	0.027	0.000	7.070	0	0.469	7.185	61.100	1.786	1.099	242	17.800	1.623	1.615	3.575
3	0.032	0.000	2.180	0	0.458	6.998	45.800	1.955	1.386	222	18.700	1.185	1.371	3.538
4	0.067	0.000	2.180	0	0.458	7.147	54.200	1.955	1.386	222	18.700	0.000	1.845	3.616

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



이상치 처리

□ 모델링 직전 이상치 처리를 결정해야 한다.

◎ 실제로 존재하는 데이터이거나 이상치 자체가 큰 효과를 갖는 경우 정제하지 않는다.

◎ 이상치의 효과를 분석 전에 알기는 어려우므로 **이상치 정제 버전과 원본 버전에 대한 모델링을 각각 수행하여 성능을 비교**하는 것이 좋다.

1 탐지 *Detection*

- 시각화: 박스플롯
- 통계 기준: IQR
- 도메인 기준: 업무 규칙에 따라 분석가가 직접 판단

2 원인 분석 *Diagnosis*

- 입력·측정 오류 여부
- 자연적으로 발생한 실제 존재하는 극단값인지 여부
- **도메인 맥락 확인**

3 처리 방법 결정 *Treatment*

- 제거: 명백한 오류 (행단위 삭제)
- **대체: 중앙값·평균·이상치 경계값**

4 검증 *Validation*

- 전·후 분포 비교
- 기술통계 재확인



/helpers/my_prep.py (step-19)

□ 이상치 정제 함수 정의

```
204 def replace_outlier(df, columns=None, method='bound', value=None, verbose=True):
205     """
206     지정한 컬럼들의 이상치를 다른 값으로 대체하는 함수
207 
208     Args:
209         df (DataFrame): 이상치를 대체할 데이터프레임
210         columns (list, optional): 이상치를 대체할 컬럼명 리스트.
211             None 이면 df 의 수치형 컬럼을 자동으로 선택한다 (기본값: None)
212         method (str): 이상치를 무엇으로 대체할지 지정한다. 대소문자는 구분하지 않는다 (기본값: 'bound')
213             - 'bound':  이상치 경계값.
214             - 'median': 해당 컬럼의 중앙값
215             - 'mean':   해당 컬럼의 평균
216             - 'value':  사용자가 value 파라미터로 지정한 고정값
217         value (number, optional): method='value' 일 때 이상치를 대체할 고정값 (기본값: None)
218         verbose (bool): 컬럼별 이상치 경계와 대체된 개수를 출력할지 여부 (기본값: True)
219 
220     Returns:
221         DataFrame: 이상치가 대체된 데이터프레임 (원본은 변경되지 않는다)
222     """
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-20)

□ 구현 단계 파이프라인

```
204  def replace_outlier(df, columns=None, method='bound', value=None, verbose=True):
205      """…
223      # --- 1) 처리 대상 컬럼 결정 ---
224
225      # --- 2) 작업 준비 ---
226
227      # --- 3) 컬럼별 이상치 판단 및 대체 ---
228
229      # --- 4) 대체 내역 출력 (정상 범위·이상치 개수·대체값) ---
230
231      # --- 5) 이상치가 대체된 데이터프레임 반환 ---
232
```



/helpers/my_prep.py (step-21)

□ 처리 준비 단계

```
223     # --- 1) 처리 대상 컬럼 결정 ---
224     # 처리 대상 컬럼 결정: 지정이 없으면 수치형 컬럼만 자동 선택
225     if columns is None:
226         columns = list(df.select_dtypes(include='number').columns)
227
228     # --- 2) 작업 준비 ---
229     result = df.copy()    # 원본을 보존하기 위해 복사본으로 작업
230     name = method.lower().strip()
231     report = []           # verbose 출력을 위해 컬럼별 대체 내역을 기록
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-22)

□ 컬럼별 이상치 처리 (1/2)

```
233     # --- 3) 컬럼별 이상치 판단 및 대체 ---
234     for c in columns:
235         # 3-1) IQR 기반 이상치 경계 계산
236         q1 = df[c].quantile(0.25)
237         q3 = df[c].quantile(0.75)
238         iqr = q3 - q1
239         lower = q1 - 1.5 * iqr
240         upper = q3 + 1.5 * iqr
241
242         # 3-2) 경계를 벗어난 값의 위치
243         is_outlier = (df[c] < lower) | (df[c] > upper)
244         count = is_outlier.sum()
245
246         # 3-3) method 에 따라 대체값을 정하고 이상치를 바꾼다
247
248             다음 단계에서 제시합니다.
249
```



/helpers/my_prep.py (step-23)

□ 컬럼별 이상치 처리 (2/2)

```
233     # --- 3) 컬럼별 이상치 판단 및 대체 ---
234     for c in columns:
235         # 3-1) TQR 기반 이상치 경계 계산
                                    ... 생략 ...
239         # 3-3) method 에 따라 대체값을 정하고 이상치를 바꾼다
240         if name == 'bound':
241             # 하한/상한 바깥의 값을 각각 하한/상한으로 잘라낸다
242             result[c] = df[c].clip(lower, upper)
243             replaced = f'{lower:.2f} 또는 {upper:.2f}'
244         else:
245             if name == 'median':
246                 # 이상치를 제외한 정상값만으로 대푯값을 구해야 이상치에 오염되지 않는다
247                 fill = df.loc[~is_outlier, c].median()
248             elif name == 'mean':
249                 fill = df.loc[~is_outlier, c].mean()
250             else:
251                 fill = value
252
253             result.loc[is_outlier, c] = fill
254             replaced = f'{fill:.2f}'
255
256         report.append([c, lower, upper, count, replaced])
```



/helpers/my_prep.py (step-24)

□ 이상치 처리 내역 출력 및 결과 반환

```
265     # --- 4) 대체 내역 출력 (정상 범위·이상치 개수·대체값) ---
266     if verbose:
267         print(f"이상치 대체 방식: '{name}' (기준: IQR x 1.5)")
268         print(f'{"컬럼":12s}{"정상 범위":>26s}{"이상치":>10s}{"대체값":>22s}')
269         print('-' * 68)
270 
271         for c, lower, upper, count, replaced in report:
272             bound = f'{lower:.2f} ~ {upper:.2f}'
273             ratio = count / len(df) * 100
274             found = f'{count}개({ratio:.1f}%)'
275             print(f'{c:12s}{bound:>24s}{found:>12s}{replaced:>20s}')
276 
277     # --- 5) 이상치가 대체된 데이터프레임 반환 ---
278     return result
```



#02. 데이터 전처리

5. 이상치 대체

◎ IQR 울타리(`$Q1 - 1.5 * IQR$ $Q3 + 1.5 * IQR$`)를 벗어난 값을 **경계값으로 대체**한다.
- 행을 삭제하지 않으므로 데이터 수(506개)가 그대로 유지된다.

◎ 경계값(`bound`)을 쓰는 이유는 **값의 순서가 보존**되기 때문이다.
- 중앙값으로 대체하면 범죄율이 가장 높은 동네가 가장 안전한 동네로 뒤바뀌어 버린다.

◎ 앞에서 로그를 취해둔 덕분에 잘려나가는 값이 크게 줄어든다.
- CRIM 을 보면 원본 기준 이상치가 66개(13.0%)였는데,
  로그 변환 후에는 6개(1.2%)만 울타리를 벗어난다.

```
1 df4 = my_prep.replace_outlier(df3, columns=feature_continuous)
2 df4.to_excel("체크포인트2_이상치대체.xlsx", index=False)
3 df4.head()
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

결과는 다음 페이지에 제시합니다.

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



#02. 데이터 전처리

5. 이상치 대체 (결과확인)

```
이상치 대체 방식: 'bound' (기준: IQR x 1.5)
컬럼                                정상 범위       이상치                 대체값
-------------------------------------------------------------------------
CRIM                        -2.12 ~ 3.74    6개(1.2%)      -2.12 또는 3.74
ZN                          -3.90 ~ 6.51    0개(0.0%)      -3.90 또는 6.51
INDUS                     -14.18 ~ 37.47    0개(0.0%)    -14.18 또는 37.47
NOX                         0.19 ~ 0.89    0개(0.0%)       0.19 또는 0.89
RM                          4.78 ~ 7.73   30개(5.9%)       4.78 또는 7.73
AGE                      -28.55 ~ 167.65    0개(0.0%)   -28.55 또는 167.65
DIS                         0.09 ~ 2.86    0개(0.0%)       0.09 또는 2.86
RAD                        -0.80 ~ 5.63    0개(0.0%)      -0.80 또는 5.63
TAX                   -301.50 ~ 1246.50    0개(0.0%) -301.50 또는 1246.50
PTRATIO                   13.20 ~ 24.40   15개(3.0%)     13.20 또는 24.40
B                           -3.38 ~ 7.01    0개(0.0%)       -3.38 또는 7.01
LSTAT                       0.85 ~ 4.11    0개(0.0%)       0.85 또는 4.11
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.006	2.944	2.310	0	0.538	6.575	65.200	1.627	0.693	296	15.300	0.000	1.788	3.219
1	0.027	0.000	7.070	0	0.469	6.421	78.900	1.786	1.099	242	17.800	0.000	2.316	3.118
2	0.027	0.000	7.070	0	0.469	7.185	61.100	1.786	1.099	242	17.800	1.623	1.615	3.575
3	0.032	0.000	2.180	0	0.458	6.998	45.800	1.955	1.386	222	18.700	1.185	1.371	3.538
4	0.067	0.000	2.180	0	0.458	7.147	54.200	1.955	1.386	222	18.700	0.000	1.845	3.616



다중 공선성 확인

□ 반복 제거 절차 (단계적 변수 제거)

◎ 변수를 하나 제거하면 나머지 모든 변수의 VIF 가 재산정되므로,
매 단계마다 VIF 를 재계산하여 가장 심한 변수 하나씩만 제거한다.

STEP 1

VIF 산출

전체 설명변수에 대해
VIF 를 일괄 계산한다.

→

STEP 2

임계값 비교

최댓값이 10 미만이면
절차 종료

→

STEP 3

최대 VIF 변수 제거

VIF 가 가장 큰 변수
한 개를 제거한다.

↺

STEP 4

반복 수행

모든 VIF 가 임계값
미만이 될 때까지
STEP 1 부터 반복.

다중공선성 처리 함수는 이미 my_prep.py에 구현되어 있습니다.

ITWILL 이광호강사



#02. 데이터 전처리

6. 다중 공선성 제거

◎ VIF 가 임계값을 넘는 변수를 하나씩 제거한다.

◎ 종속변수(MEDV)는 대상에서 빼야 하므로 연속형 독립변수만 지정한다.

```
1 df5 = my_prep.reduce_vif(df4, columns=feature_continuous, threshold=10.0)
2 df5.to_excel("체크포인트3_다중공선성제거.xlsx", index=False)
3 df5.head()
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

```
완료! 남은 변수: ['AGE', 'B', 'CRIM', 'DIS', 'INDUS', 'LSTAT', 'NOX', 'PTRATIO', 'RAD', 'RM', 'TAX', 'ZN']
최대 VIF = 7.08
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.006	2.944	2.310	0	0.538	6.575	65.200	1.627	0.693	296	15.300	0.000	1.788	3.219
1	0.027	0.000	7.070	0	0.469	6.421	78.900	1.786	1.099	242	17.800	0.000	2.316	3.118
2	0.027	0.000	7.070	0	0.469	7.185	61.100	1.786	1.099	242	17.800	1.623	1.615	3.575
3	0.032	0.000	2.180	0	0.458	6.998	45.800	1.955	1.386	222	18.700	1.185	1.371	3.538
4	0.067	0.000	2.180	0	0.458	7.147	54.200	1.955	1.386	222	18.700	0.000	1.845	3.616

데이터 표준화

□ 연속형 변수 → 스케일링(표준화)

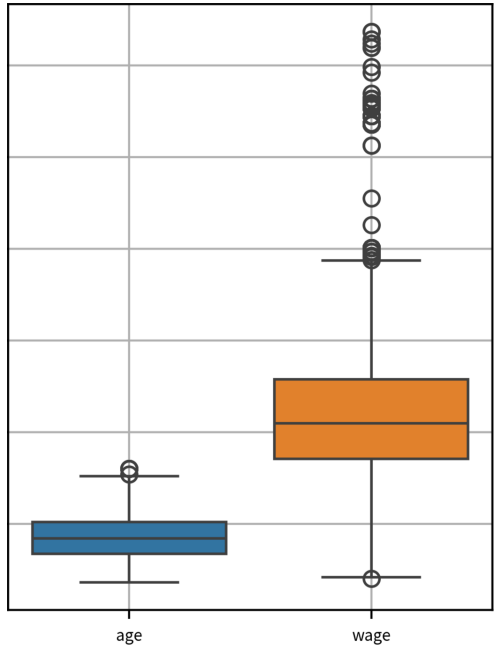
◎ 연속형 변수의 크기(단위·범위)를 조정하여 모델의 학습 안정성 향상

◎ 변수 단위(범위)가 다른 경우 일부 분석 모델이 큰 값 위주로 학습하기 때문에 분석 결과에 영향을 줌.

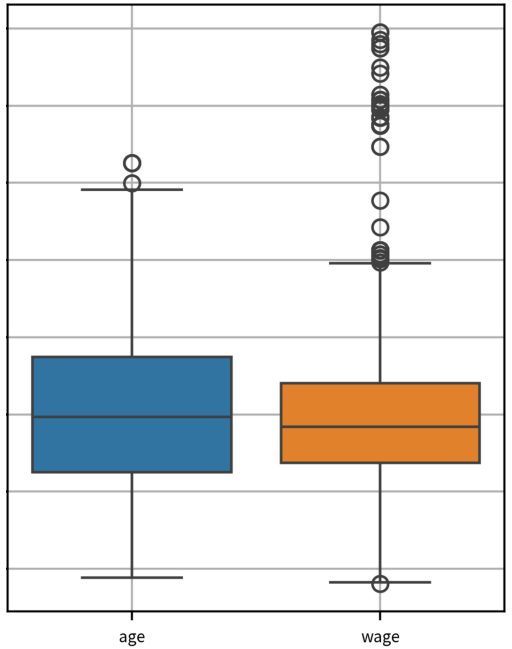
◎ 그러므로 데이터의 스케일(단위, 척도)를 비슷하게 맞춰서 골고루 학습하게 하려는 것이 목적

- 탐색적 데이터 분석 단계에서는 불필요한 전처리.
- 주로 StandardScaler를 사용

age	wage
18	75.043154
24	70.476020
45	130.982177
43	154.685293
50	75.043154
...	...
27	87.981033
55	90.481913



age	wage
-2.1	-0.9
-1.6	-1
0.2	0.5
0.1	1.0
0.7	-0.9
...	...
-1.3	-0.6
1.1	-0.5



ITWILL 이광호강사



/helpers/my_prep.py (step-25)

□ 데이터 표준화 함수 정의

```
413 def scaling(df, columns=None, method='standard', save_path=None, verbose=True):
414     """
415     지정한 컬럼들의 값의 범위(스케일)를 통일하는 함수
416 
417     Args:
418         df (DataFrame): 스케일링을 적용할 데이터프레임
419         columns (list, optional): 스케일링할 컬럼명 리스트.
420             None 이면 df 의 수치형 컬럼을 자동으로 선택한다 (기본값: None)
421         method (str): 사용할 스케일러 이름. 대소문자와 뒤의 'Scaler' 는 무시하므로
422             'standard', 'Standard', 'StandardScaler' 를 모두 같게 취급한다 (기본값: 'standard')
423             - 'standard': (x - 평균) / 표준편차 -> 평균 0, 표준편차 1
424             - 'minmax':   (x - 최소) / (최대 - 최소) -> 0 ~ 1
425             - 'robust':   (x - 중앙값) / IQR -> 이상치의 영향을 덜 받음
426             - 'maxabs':   x / |최대| -> -1 ~ 1 (0을 보존하므로 희소 데이터에 사용)
427         save_path (str, optional): 학습된 스케일러를 저장할 pkl 파일 경로
428             (예: 'models/scaler.pkl') (기본값: None, 저장하지 않음)
429         verbose (bool): 컬럼별 스케일링 전후의 값의 범위를 출력할지 여부 (기본값: True)
430 
431     Returns:
432         DataFrame: 스케일링이 적용된 데이터프레임 (원본은 변경되지 않는다)
433     """
```


ITWILL 이광호강사

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-26)

□ 작업 단계 파이프라인

```
413 def scaling(df, columns=None, method='standard', save_path=None, verbose=True):
414     """…
434     # --- 1) 처리 대상 컬럼 결정 ---
435
436     # --- 2) 작업 준비 및 스케일러 이름 정규화 ---
437
438     # --- 3) 스케일러 학습 및 변환 ---
439
440     # --- 4) 변환 내역 출력 (컬럼별 값의 범위 변화) ---
441
442     # --- 5) 학습된 스케일러 저장 (선택) ---
443
444     # --- 6) 스케일링된 데이터프레임 반환 ---
445
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-27)

□ 대상 컬럼 식별 및 스케일링 종류 판별

```
434     # --- 1) 처리 대상 컬럼 결정 ---
435     # 처리 대상 컬럼 결정: 지정이 없으면 수치형 컬럼만 자동 선택
436     if columns is None:
437         columns = list(df.select_dtypes(include='number').columns)
438
439     # --- 2) 작업 준비 및 스케일러 이름 정규화 ---
440     result = df.copy()    # 원본을 보존하기 위해 복사본으로 작업
441
442     # 대소문자와 뒤에 붙은 'scaler' 를 떼어내 이름을 통일한다 ('StandardScaler' -> 'standard')
443     name = method.lower().replace('scaler', '').strip()
444
445     # 오타를 냈을 때 KeyError 대신 사용 가능한 이름을 알려준다
446     if name not in SCALERS:
447         raise ValueError(f"지원하지 않는 스케일러입니다: '{method}' "
448                          f"(사용 가능: {list(SCALERS.keys())})")
```

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



/helpers/my_prep.py (step-28)

□ 스케일링 처리 및 처리 내역 출력

```
450     # --- 3) 스케일러 학습 및 변환 ---
451     # 이름에 해당하는 클래스로 스케일러를 만들어 대상 컬럼의 기준값(평균, 표준편차 등)을 학습시킨다
452     scaler = SCALERS[name]()
453     result[columns] = scaler.fit_transform(df[columns])
454
455     # --- 4) 변환 내역 출력 (컬럼별 값의 범위 변화) ---
456     if verbose:
457         print(f'{type(scaler).__name__} 적용 ({len(columns)}개 컬럼)')
458         print(f'{"컬럼":12s}{"변환 전":>22s}{"변환 후":>22s}')
459         print('-' * 56)
460
461         for c in columns:
462             before = f'{df[c].min():.2f} ~ {df[c].max():.2f}'
463             after = f'{result[c].min():.2f} ~ {result[c].max():.2f}'
464             print(f'{c:12s}{before:>20s}{after:>22s}')
```



/helpers/my_prep.py (step-29)

□ 스케일러 객체 저장 및 처리 결과 반환

```
466     # --- 5) 학습된 스케일러 저장 (선택) ---
467     # 스케일러 안에는 train 에서 구한 기준값(평균·표준편차 등)이 들어 있으므로,
468     # 이 파일이 있어야 test 데이터에 똑같은 기준을 적용할 수 있다
469     if save_path:
470         folder = os.path.dirname(save_path)
471         if folder:
472             os.makedirs(folder, exist_ok=True)    # 경로에 없는 폴더가 있으면 만들어 준다
473 
474         joblib.dump(scaler, save_path)
475 
476         if verbose:
477             print(f'\n스케일러 저장: {save_path} ({type(scaler).__name__})')
478 
479     # --- 6) 스케일링된 데이터프레임 반환 ---
480     return result
```



#02. 데이터 전처리

7. 정규화

◎ 연속형 독립변수를 대상으로 한다.

◎ **정규화는 OLS 의 R^2를 바꾸지 못한다.**

- 각 변수에 상수를 더하고 곱하는 선형 변환일 뿐이고,
  회귀계수가 그 반대로 조정되면서 예측값이 완전히 동일해지기 때문이다.
- Ridge·Lasso 처럼 계수에 벌점을 주는 모델을 쓸 때다.

```
1 df6 = my_prep.scaling(df5, columns=feature_continuous, method='standard')
2 df6.to_excel("체크포인트4_정규화.xlsx", index=False)
3 df6.head()
```

✓ 마지막 실행: 오전 11:35:29, 기간: 0.0s Python

1. 모델링용 전처리
2. Boston Housing 회귀 모델 적합
3. Diamond Price Prediction 회귀 모델 적합



#02. 데이터 전처리

7. 정규화 (결과 확인)

```
StandardScaler 적용 (12개 컬럼)
컬럼                                변환 전                  변환 후
-----------------------------------------------------------------
CRIM                          0.01 ~ 3.74          -0.80 ~ 2.91
ZN                            0.00 ~ 4.62          -0.59 ~ 2.26
INDUS                         0.46 ~ 27.74         -1.56 ~ 2.42
NOX                           0.39 ~ 0.87          -1.47 ~ 2.73
RM                            4.78 ~ 7.73          -2.38 ~ 2.31
AGE                           2.90 ~ 100.00        -2.34 ~ 1.12
DIS                           0.76 ~ 2.57          -1.75 ~ 2.65
RAD                           0.69 ~ 3.22          -1.81 ~ 1.55
TAX                         187.00 ~ 711.00        -1.31 ~ 1.80
PTRATIO                       13.20 ~ 22.00        -2.46 ~ 1.65
B                             0.00 ~ 5.99          -1.18 ~ 2.23
LSTAT                         1.00 ~ 3.66          -2.73 ~ 2.20
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	-0.796	1.229	-1.288	0	-0.144	0.476	-0.120	0.357	-1.812	-0.667	-1.477	-1.184	-1.276	3.219
1	-0.776	-0.589	-0.593	0	-0.740	0.231	0.367	0.742	-1.273	-0.987	-0.310	-1.184	-0.295	3.118
2	-0.776	-0.589	-0.593	0	-0.740	1.445	-0.266	0.742	-1.273	-0.987	-0.310	-0.257	-1.597	3.575
3	-0.771	-0.589	-1.307	0	-0.835	1.148	-0.810	1.150	-0.890	-1.106	0.110	-0.507	-2.051	3.538
4	-0.736	-0.589	-1.307	0	-0.835	1.384	-0.511	1.150	-0.890	-1.106	0.110	-1.184	-1.170	3.616



연습문제 보험료 가격 결정 구조 데이터셋의 모델링용 전처리

☐ insurance 데이터셋에 대한 모델링용 전처리를 수행하세요.

- **insurance_qtcheck** : 보험 비용 데이터셋의 품질 검사 완료 버전
- **insurance_qtcheck_desc** : 보험 비용 데이터셋의 연속형 변수에 대한 기술 통계량
- **insurance_qtcheck_category_desc** : 보험 비용 데이터셋의 범주형 변수에 대한 기술 통계량